Field Hours

CSCE 652 Software Reverse Engineering

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Sections before opcodes

You start reverse engineering at the object file the compiler emitted, not at a guess about the source. The mechanism is the section table. You read 3 section names before any opcode: code, initialized data, and relocations. If the code section is 40 bytes, the machine text is 40 bytes, and every later offset must fall inside that length.

You do not take a function location from a name the linker may rewrite. A symbol entry stores a name, a section index, and a value that is an offset inside that section. When the value is 16 and the section is code, byte 16 of the code is the first byte of that function. CSCE 313 is the systems prerequisite. You check this on 1 object until each offset lands on the bytes you see.

Three names you read before any opcode.

Code section

Initialized data

Relocation records

A symbol at offset 16

1. The code section is 40 bytes long. You read the symbol table before you decode.

2. One symbol has value 16 and its section index selects code. Its first byte is byte 16 of that section.

3. Offset 16 is inside a length of 40, so the symbol names a real location. You do not move it to match a source name.

Problem 1. The code section is 40 bytes. A symbol's value is 36 and its section is code. Is byte 36 inside the section, and how many bytes of the section remain starting at that byte?

Answer. Yes. 40 minus 36 is 4, so 4 bytes remain, counting byte 36. An offset of 40 itself would be outside the section.

Problem 2. The linker will rewrite a local name, and the symbol value is still 16 in the code section. Where is the function?

Answer. Byte 16 of the code section. The name is not the location. CSCE 313 told you a process image is more than a listing; here the listing starts at the section offset.

Lecture 2 · 90 minutes

Headers tell you where to read

You parse a file format by walking headers, which is the mechanism, not by scanning for a readable string. An executable header stores the entry point and the file offset of each loadable segment. Suppose the header occupies bytes 0 through 63, so its size is 64, and the first segment starts at offset 64. Byte 64 is the first mapped byte. You write both numbers down before you disassemble.

A section table is a second walk from a base the file header gives you. If each section header is 40 bytes and the table starts at offset 200, section 2 begins at 200 plus 80, which is 280. You read the header at 280 and only then trust its fields. A string you noticed at offset 12 is not the entry point. Two offsets, 200 and 280, are enough to show you did the addition.

From the file header to one section.

Read the table base -> Multiply index by width -> Add base to that product -> Read the header there

Section 2 at a 40-byte stride

1. The section table starts at file offset 200. Each section header is 40 bytes wide.

2. Index 2 contributes 2 times 40, which is 80. You add that to the base: 200 plus 80 is 280.

3. You parse the header at offset 280. The string at offset 12 stays a string, not an entry point.

Problem 1. The section table starts at offset 120. Each header is 40 bytes. Where do you read section 3?

Answer. At offset 240. 3 times 40 is 120, and 120 plus 120 is 240.

Problem 2. The file header occupies bytes 0 through 63, and the first loadable segment starts at offset 64. Which byte is the first mapped byte?

Answer. Byte 64. Bytes 0 through 63 are the header you already consumed. A readable string in those bytes is not the entry point.

Lecture 3 · 90 minutes

Decode a fixed width

You disassemble by consuming a stated width and then decoding the opcode, which is the mechanism. In this toy encoding every instruction is 2 bytes: opcode, then a register or a small immediate. A buffer of 8 bytes holds exactly 4 instructions when you start at offset 0. If you start at offset 1 you split two real instructions and invent a byte pair nobody issued. Start only at a header entry or a branch target.

You keep a short opcode table and you refuse bytes that are not in it. Opcode 1 is add, opcode 2 is jump, and opcode 3 is return. The pair 02 05 is a jump whose operand field is 5, not an add. You write one row per instruction: address, name, operand. Four rows cover the 8-byte buffer. You do not run the bytes. A row is not proof the program reached that address.

Where you are allowed to start a decode.

Start at offset 0

Width 2 consumes pairs

8 bytes give 4 rows

Opcodes stay in the table

Start at offset 1

Pairs split real instructions

The new pairs were not issued

Refuse that start

Four rows from eight bytes

1. The buffer is the 8 bytes starting at a named offset. Each instruction consumes 2 bytes, so you expect 4 rows.

2. The bytes 02 05 become a jump with operand 5, because opcode 2 is jump. Opcode 1 would have been add.

3. You stop after the fourth row. You do not execute the buffer, and you do not start the next decode at an odd offset.

Problem 1. The toy width is 2 bytes and the buffer is 8 bytes, starting at a named offset of 0. How many instruction rows do you write?

Answer. 4. 8 divided by 2 is 4. Each row is address, name, and operand. You do not run the bytes.

Problem 2. You began the same 8-byte buffer at offset 1. What is wrong with the four pairs you see?

Answer. They are not the issued instructions. An odd start splits two real 2-byte instructions. You only decode from a header entry or a branch target.

Lecture 4 · 90 minutes

Edges the instruction named

A control-flow graph is the mechanism that groups decoded instructions into basic blocks. A block is a straight run that ends at a branch, a call, or a return. A function with 12 instructions and 3 branches often yields 4 blocks: entry, two targets, and a fall-through. You draw an edge only when some instruction names that destination. A curve you added because the code looks like a loop is not an edge.

You label blocks so a later note can cite them. Call the entry B0. A conditional jump at the end of B0 has two successors, so the out-degree of B0 is 2. An indirect jump through a register is not those 2 edges. You mark it unresolved until a run reports one concrete target. One unresolved edge is a property of your graph. You do not replace it with an address you hoped for.

One block ends only at a transfer.

1. End the block at a branch

2. Read the named targets

3. Draw only those edges

4. Mark indirect edges open

Out-degree 2 at B0

1. B0 ends in a conditional jump. The instruction names two successors, B1 and B2.

2. You draw those 2 edges and no others. The out-degree of B0 is 2.

3. A later indirect jump stays unresolved. You do not add a third edge because the shape reminds you of a loop.

Problem 1. A function has 12 instructions, and 3 of them are branches. You drew 4 blocks but added a back edge no instruction names. Which edge do you delete?

Answer. The back edge you invented. An edge exists only when an instruction names the destination. Looking like a loop is not a name.

Problem 2. B0 ends in an indirect jump through a register. How many resolved outgoing edges do you record before a run?

Answer. 0 resolved edges. Two successors would be a conditional jump that names both targets. An indirect target stays unresolved until a run reports one concrete landing block.

Lecture 5 · 90 minutes

Facts you can claim without running

Static analysis walks the graph without running the program. The mechanism here is constant propagation inside one block. If B0 writes the immediate 7 into a register and no later instruction in B0 overwrites it, a use at the end of B0 still sees 7. You stop that claim at a join. Two predecessors means you carry 7 forward only when both predecessors carry 7. Otherwise the value at the join is unknown, not 7.

You also read calls off the graph. A direct call edge names exactly 1 callee. If you see 5 call sites and 4 of them name the same function, you have one helper, not 5 unrelated functions. You list callees per block. A function with 0 call edges is a leaf. You do not treat a leaf as harmless: it can still write the byte the caller passed in a register. The leaf is small. The effect is not.

When the constant 7 may cross a join.

One predecessor

B0 wrote immediate 7

No later overwrite in B0

The use still sees 7

Two predecessors

Carry 7 only if both do

A disagreement blocks the claim

The join stays unknown

Five call sites, one helper

1. You list direct call edges. Each edge names exactly 1 callee.

2. Five sites appear, and 4 of them name the same function. You record one helper used from 4 sites, plus the fifth callee.

3. A different function has 0 call edges. You mark it a leaf and you still ask what byte it writes.

Problem 1. B0 carries the constant 7. One predecessor of the join also carries 7, and the other carries an unknown value. What do you claim at the join?

Answer. The value is unknown, not 7. You need both predecessors to carry 7 before you propagate it. One agreement is not a join fact.

Problem 2. You see 5 direct call sites and 4 name the same function. How many distinct callees do those 4 sites contribute?

Answer. 1 helper. The fifth site may name someone else, so the function count is not 5. A leaf, with 0 call edges, is separate and is not automatically harmless.

Lecture 6 · 90 minutes

Counters from one run

Dynamic analysis runs the program and records facts the static graph left open. The mechanism is a counter on each basic block. After 1 run that takes the true side of a branch, the counter on B1 is 1 and the counter on B2 is 0. A count of 0 means this run missed the block. It does not mean the block is dead code. You need a second input before you delete B2.

You add an indirect edge only when a trace shows the landing block. If B0 lands in B3 on this run, you add that single edge and you tag it as input 2 of 6. You do not add the other targets you have not seen. A watch on a register is the same mechanism at one instruction: you record the value you observed, such as 5, beside the address. The address without the value is an incomplete note.

What one run is allowed to add.

Count entries per block -> Keep a zero count -> Add only seen edges -> Store value and address

Input 2 lands in B3

1. This is input 2 of 6. The indirect jump in B0 lands in B3, so you add that one edge and tag it with input 2.

2. B2's counter stays 0 on this run. You leave B2 in the graph.

3. A register watch at the jump records the value 5 and the instruction address. You do not fill in targets from the other 4 inputs you have not run.

Problem 1. After 1 run, B1 has count 1 and B2 has count 0. May you delete B2?

Answer. No. A count of 0 means this run missed B2. Dead code is a stronger claim, and you need another input before you delete the block.

Problem 2. On input 2 of 6 the indirect jump lands in B3. How many new indirect edges do you add from that fact?

Answer. 1 edge, tagged as input 2. You do not add the landings you have not seen among the other inputs. A register value such as 5 is recorded beside its address, not turned into extra edges.

Lecture 7 · 90 minutes

Live blocks and a measured file

Obfuscation keeps behavior and hides the graph. One mechanism inserts a bogus block on a predicate you can prove is always false. If the live function had 4 blocks and the transform adds 2 bogus blocks, a raw count of 6 overstates the live graph. You mark a block live only when a solved predicate or a trace reaches it. Dead blocks stay on the drawing, and you label them dead so you do not analyze them as reachable.

You describe a suspicious file by measurements, not by a nickname. You record 3 facts: the file hash, the entry-point offset, and the number of unresolved indirect edges. A file with 9 unresolved edges is harder to summarize than a file with 1. You do not run that file on the machine that stores your notes. A writeup that omits the hash cannot be tied back to the bytes you read.

A raw block count is not the live graph.

Raw drawing

4 real blocks plus 2

Count reads 6

Bogus predicate is false

Live subset

Reachable by proof or trace

Dead blocks stay labeled

Do not treat them as live

Six blocks, four live

1. The transform added 2 bogus blocks to a function that had 4. The drawing shows 6.

2. The predicate into each bogus block is always false. You label those 2 dead.

3. You summarize the file with its hash, its entry-point offset, and 9 unresolved indirect edges. You do not execute it on the notes machine.

Problem 1. A function's drawing has 6 blocks. You can show 2 of them sit under a predicate that is always false. How many blocks are live?

Answer. 4. The raw count 6 overstates the live graph by the 2 bogus blocks. Dead blocks stay on the drawing and stay labeled dead.

Problem 2. Your note lists 9 unresolved edges and an entry-point offset, but no hash. What can a

reader not do?

Answer. They cannot tie the note to the bytes you read. The hash is 1 of the 3 required facts, with the entry-point offset and the unresolved-edge count. You also do not run the file on the machine that stores the notes.

Lecture 8 · 90 minutes

What you can now recover

You can now move from a file header to a control-flow graph you can defend. You can compute a section location, 200 plus 2 times 40, and land on byte 280. You can decode a 2-byte toy instruction and refuse an edge the instruction did not name. Propagation of the constant 7 stops at a join with a disagreeing predecessor. A dynamic count of 0 means that run missed the block, not that the block is absent.

The lab path is Ali Hadi's public reverse-engineering sequence, with pwn.college as the place you practice. Spend the hours on these 8 mechanisms, not on a tour of menus. Your closing exercise is 1 file: parse the header, decode 4 instructions, draw the blocks, and name the 1 edge that is still unresolved. If the section-offset addition is missing, this course is not finished.

The close of this course is one file you can defend.

1. Add the section offset

2. Decode four instructions

3. Draw only named edges

4. Name one open edge

Land on byte 280, then stop

1. The section table base is 200 and the header width is 40. Index 2 yields 80, and 200 plus 80 is 280.

2. From a named start you decode 4 instructions of width 2. You draw an edge only when the instruction names it.

3. You leave 1 indirect edge unresolved and you say so. The lab hours belong to Ali Hadi's sequence and to pwn.college, on mechanisms like this one.

Problem 1. The section table is at offset 200, each header is 40 bytes, and you want section 2. Which file offset do you read, and what is the product you added?

Answer. You read offset 280. The product is 80, because 2 times 40 is 80, and 200 plus 80 is 280. Without that addition this course is not finished.

Problem 2. Your closing file shows a dynamic count of 0 on B2 and a join where only one predecessor carries the constant 7. What may you claim?

Answer. This run missed B2, and the join value is unknown rather than 7. A zero count is not absence. One predecessor is not both predecessors.